Proof Without the Answer

A six-week project in graph coloring and zero-knowledge proofs

Team proposal | Three community-college students | September 2026

The question: Can someone convince us that a puzzle has a valid solution while keeping the complete solution private? How often could an invalid attempt survive a sequence of random checks?

We will build a small Python experiment around the classic graph 3-coloring protocol. A graph describes which items conflict; a coloring assigns one of three labels to each item. The prover keeps the coloring, and the verifier checks one randomly selected edge at a time. Our contribution will be a clear implementation, short proofs, controlled experiments, and an explanation that people outside mathematics can follow.

What we will finish

Deliverable	What someone can see or do
A reproducible Python project	Generate or load graphs, try honest and dishonest provers, and replay a verification round.
A small experimental study	Compare measured acceptance rates with mathematical predictions, including uncertainty.
A short paper and visual explanation	Read a 4-6 page student technical report, inspect two main figures, and watch a 2-minute demo.

A manageable pace

Plan for **3-4 hours per person per week**, including one 30-45 minute online meeting. That is about 18-24 hours each over six weeks. Some setup may take longer if Python or Git is new. Keep optional work optional; a clean notebook is enough for the demonstration.

The core ends at a working experiment and a readable report. A website, image recognition, blockchain integration, and a production cryptography library are outside the six-week scope. No prior cryptography is required. Basic Python, algebra, and a willingness to ask questions are enough.

Read this first

Sections 1-5 explain the ideas. Sections 6-9 specify what to build and measure. Sections 10-12 cover the schedule, paper, release, and references. This is a proposal: numerical illustrations are theoretical predictions, not completed project results.

1. The puzzle, in plain language

A **graph** is a collection of vertices (dots) and edges (connections). It is not an x-y plot. Imagine that each vertex is a class and each edge joins two classes that cannot share a time slot. Three colors represent three possible slots.

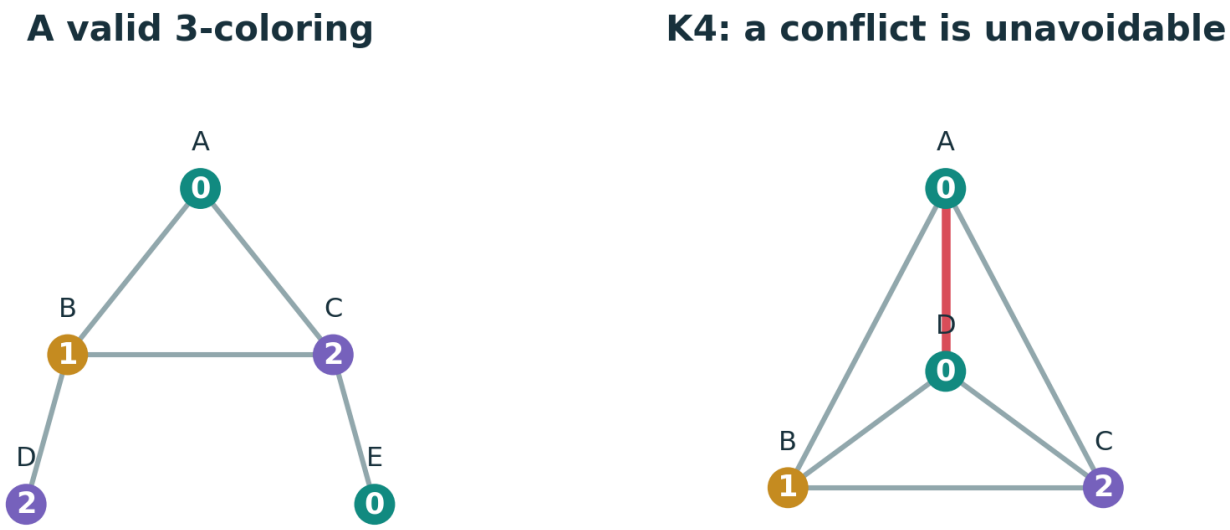


Figure 1. Left: a triangle with two attached vertices, correctly colored. Right: K4, where every pair of four vertices is connected. With only three colors, at least one connected pair must match. Letters identify vertices; numbers identify colors.

Term	Meaning in this project
Proper 3-coloring	Give every vertex a label 0, 1, or 2, with different labels at the ends of every edge. Using fewer than three labels is allowed.
Public statement	“This graph has a proper 3-coloring.” Everyone knows the graph.
Witness	One complete valid coloring, held by the prover.
Prover / verifier	The party supporting the statement / the party checking it.
Commitment	A sealed value: it should hide a color and prevent changing it after the challenge.
Transcript	The commitments, selected edge, and two revealed openings from an interaction.

Two different jobs

Check a supplied coloring: inspect its edges and find conflicts. This takes time proportional to the graph size. **Find a coloring:** search for labels that satisfy every constraint. This can be much harder. Graph 3-colorability is an NP-complete problem; explaining the distinction is enough for this project. We will not prove NP-completeness. [1, 2]

The scheduling story is an analogy. Our experiment demonstrates a general verification idea; it does not build a confidential scheduling service.

2. One round of the protocol

Keep the graph and vertex IDs fixed throughout the session. The prover changes the **names of the three colors**, not the vertex identities. Use one random permutation of the palette for the whole graph.

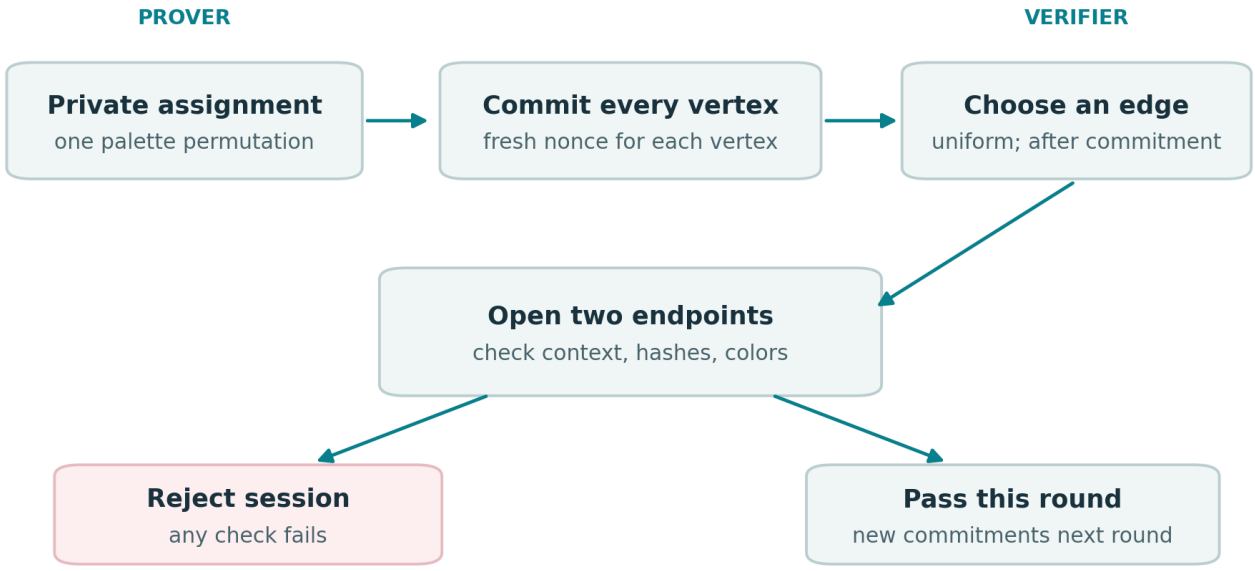


Figure 2. The verifier receives all commitments before choosing a challenge. Only the selected endpoints are opened. The full coloring belongs to the prover's view and the educational debug view.

1. **Prepare:** the prover starts with a valid or deliberately invalid assignment. Randomly choose one of the six permutations of labels 0, 1, and 2.

2. **Commit:** apply that permutation to all vertices. Create a separate commitment for every vertex using fresh independent randomness. Send the complete commitment list.

3. **Challenge:** after receiving that list, the verifier samples one edge uniformly from all edges. Sampling is independent across rounds, with replacement.

4. **Open:** the prover reveals the two permuted colors and the randomness needed to open those two commitments.

5. **Verify:** check the challenged endpoints, the allowed color range, both commitment openings, and that the two colors differ. Reject if any check fails.

6. **Repeat:** discard the round state. Use a fresh palette permutation and fresh randomness in the next round. Accept the session only if every one of its t rounds passes. [2, 3]

Rules worth enforcing in code

• Never choose the edge before the commitments are fixed.

• Never treat different hash values as evidence of different colors; compare the opened colors.

• Open at most one edge per commitment batch. Reject a second opening request on that batch.

• Give the verifier only public messages. A separate experiment controller can know the ground truth.

• Reject malformed graph files. An edgeless graph is trivially colorable; label it as such and skip edge sampling.

3. The mathematical claims we can prove

Let **m** be the number of edges, **b** the number of edges whose endpoints have the same color in a fixed assignment, and **t** the number of rounds. Assume binding commitments, valid encodings, and independent uniform challenges after each commitment batch.

Claim A: an honest prover always passes

A proper coloring gives different labels on every edge. A permutation preserves whether two labels are equal. Honest openings match their commitments. Therefore every possible edge challenge passes. This is **completeness**, under correct execution of the protocol.

Claim B: a fixed invalid assignment has a predictable acceptance rate

There are b failing edges among m equally likely choices. One round passes with probability 1 - b/m. If each round uses the same assignment up to palette permutation and independent challenges, all t rounds pass with probability:

$$p_t = \left(1 - \frac{b}{m}\right)^t$$

The probability of detecting at least one conflict is 1 - p_t. For K4, the assignment (0, 1, 2, 0) has exactly one bad edge among six. At t = 20, its theoretical acceptance probability is **2.61%**.

Claim C: a false graph statement has a worst-case bound

If the graph is not 3-colorable, every assignment has at least one bad edge. Even when the prover changes its assignment between rounds, each conditional round passes with probability at most 1 - 1/m, assuming its commitments bind it before the next challenge. Multiplying these conditional bounds gives:

$$\Pr[\text{accept} \mid G \notin \text{3COL}] \leq \left(1 - \frac{1}{m}\right)^t$$

This is the protocol's **soundness** argument under the commitment assumptions. A concrete hash implementation has computational assumptions beyond this elementary calculation. [2, 3]

A distinction that keeps the paper accurate

An invalid coloring of a colorable graph tests detection of a **bad supplied witness**. It is not a false public statement: a different valid coloring may exist. Use K4-based graphs for actual false-statement experiments. Also, “2.61% acceptance under this cheating model” is not “a 97.39% probability this person is honest.”

4. Why the verifier does not receive the full answer

Take a valid edge and order its endpoints by vertex ID. Its two colors are distinct. Under a uniformly random palette permutation, the observed pair is equally likely to be any of:

(0,1), (0,2), (1,0), (1,2), (2,0), (2,1).

Each outcome has probability 1/6, regardless of which valid coloring the prover holds. This is a short counting proof: one permutation maps the original ordered pair to each possible distinct pair.

A small simulation argument

For a verifier that samples an independent uniform edge, an offline simulator can choose that edge, choose one of the six distinct pairs, commit to those two labels, fill other vertices with arbitrary committed labels, and output only the two selected openings. It does not need a valid coloring. Under an ideal hiding commitment, its visible view matches the real one; with computationally hiding commitments, the comparison becomes computational indistinguishability. [2, 4]

The crucial point is **simulation without the witness**, not merely “we did not print the answer.” The elementary pair-counting argument does not, by itself, prove security of our hash code or security against every malicious verifier. The paper will give an honest-verifier proof sketch and cite the standard general result. Full malicious-verifier proofs are optional reading.

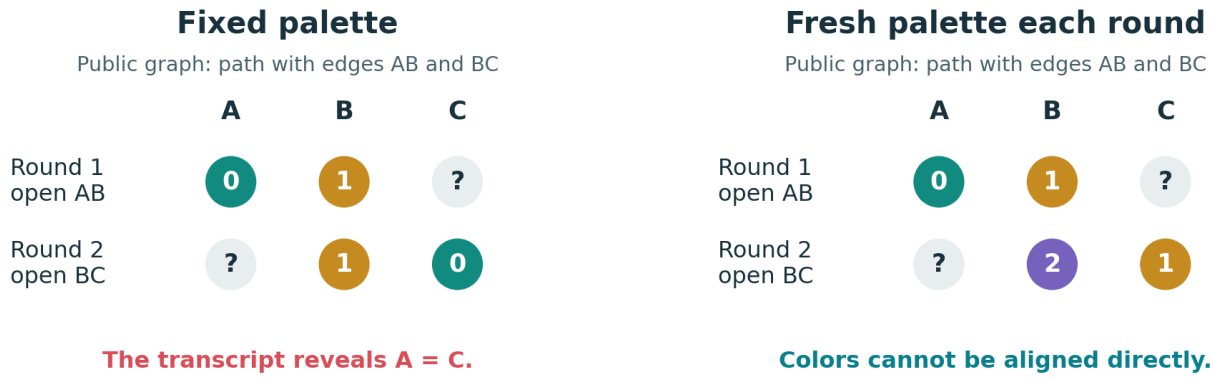


Figure 3. With a fixed palette, partial observations can be joined into a larger coloring. Fresh independent palette permutations break that direct link. Fresh commitment randomness is also required; changing only the permutation is insufficient.

What “zero knowledge” does not promise

The graph is public. Someone may already be able to solve a small or easy graph. The intended guarantee is that the interaction adds no efficiently obtainable information beyond the public statement, under the model's assumptions. A histogram that looks uniform is an illustration, not a proof of zero knowledge. A saved accepting transcript is also not independent proof of a live, unpredictable challenge.

5. A small research question of our own

What happens if we add valid constraints around the same impossible core? Start with K4 and attach L new leaf vertices to one core vertex. Each leaf has one edge and can receive a color different from its neighbor.

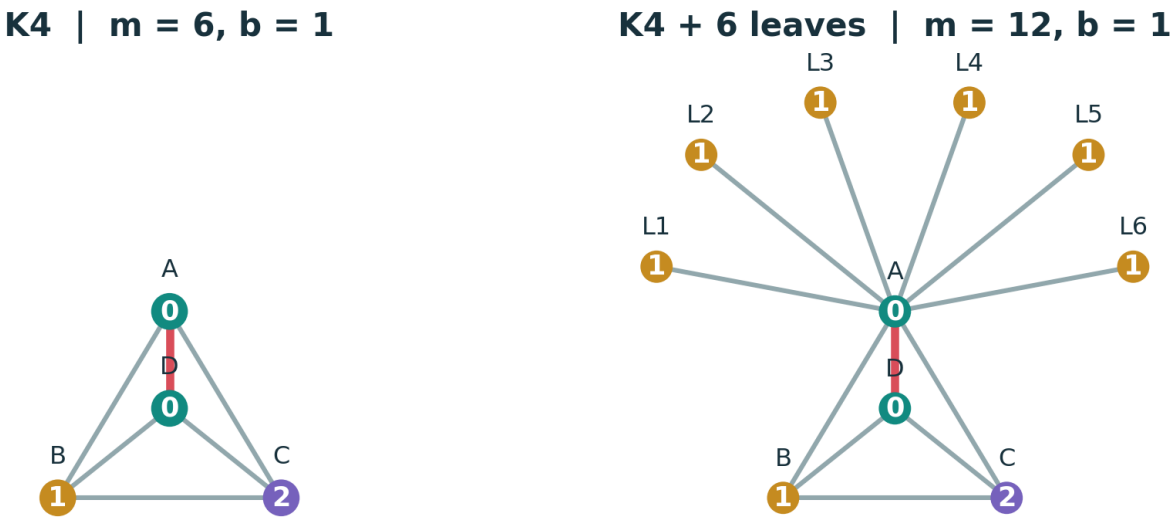


Figure 4. The red edge remains the only conflict in our chosen assignment. The added leaf edges are all satisfied. The original K4 still prevents a proper 3-coloring of the whole graph.

A short proof of the construction

K4 needs four different labels because every pair is adjacent. By the pigeonhole principle, any assignment of three labels to four vertices repeats a label on an edge. Our assignment (0,1,2,0) produces exactly one such edge. Adding properly colored leaves neither removes this conflict nor creates another one. Thus the minimum number of bad edges is exactly one, while m = 6 + L.

Added leaves L	Total edges m	Acceptance after 20 rounds	Rounds needed for at most 1% acceptance
0	6	2.61%	26
6	12	17.55%	53
18	24	42.69%	109

All numbers in this table are theoretical predictions, computed from (1 - 1/m)^t. They are not experimental findings.

This gives the paper a specific question, a proof, and a visible effect: at a fixed number of uniformly random checks, more edges can make the same local defect harder to catch. The important quantity for a fixed assignment is the **fraction of violated constraints, b/m**. This is our chosen experimental construction, not a claim of a newly discovered theorem or a new attack on ZKP systems.

6. The program we will build

Use Python with NetworkX for graph handling, Matplotlib for figures, and a notebook for the demo. NumPy and SciPy can help with experiments. Use ordinary Python modules for the protocol so a notebook is not required to run the core.

Component	Minimum behavior
Graph input	Load JSON containing explicit vertices and an edge list. Validate IDs, duplicate edges, and self-loops.
Graph generation	Make a triangle, K4, K4 with leaves, and planted colorable graphs.
Coloring checker	Return valid/invalid and the list of monochromatic edges for a supplied assignment.
Small exact solver	Backtracking search with TRUE, FALSE, or UNKNOWN status; return a witness when found.
Prover	Honest witness mode; fixed invalid assignment mode; an opening-tampering mode.
Verifier	Process commitments and two openings without access to the private coloring.
Experiment runner	Repeat independent sessions; write outcomes and configuration; produce plots.
Demo notebook	Show a graph, sealed vertices, a chosen edge, its openings, and the running decision.

What “recognize a graph” means here

It means **load a structured graph file and inspect its mathematical properties**. Reading a hand-drawn or photographed graph is a separate computer-vision project and can wait. An adjacency-matrix importer is an easy optional extension.

Generate valid examples without solving a hard problem

First assign each vertex a hidden label. Then add edges only between differently labeled vertices. This guarantees a witness by construction. Use a random partition with all three groups nonempty and independent edge sampling across groups. These are planted examples, not uniformly random samples of all colorable graphs.

Search honestly

For imported tiny graphs, recursively try labels 0, 1, 2; undo a choice when it conflicts with an already colored neighbor. Start with at most 12 vertices and a node-expansion budget. Exhausting the search proves FALSE; hitting the budget means UNKNOWN. A greedy coloring using four colors does **not** prove that three colors are impossible. Greedy routines are heuristics. [5]

7. Interfaces and commitment details

Keep the data simple

Public graph example:

```
{"vertices":[0,1,2],"edges":[[0,1],[1,2],[0,2]]}
```

Private witness example:

```
{"colors":{"0":0,"1":1,"2":2}}
```

Normalize integer IDs and undirected edges before hashing or sampling. The graph serializer should have a single canonical representation. Preserve isolated vertices. Reject duplicate edges instead of accidentally giving them extra challenge weight. Reject self-loops as unsupported input in this project.

```
make_planted(n, edge_probability, graph_rng) -> (graph, witness)
check_coloring(graph, colors) -> (is_valid, bad_edges)
solve_three_color(graph, search_budget) -> (status, witness_or_none)
commit_round(graph, colors, prover_rng) -> (public_batch, private_state)
choose_edge(graph, verifier_rng) -> edge
open_edge(private_state, edge) -> two_openings
verify_round(graph, public_batch, edge, two_openings) -> decision
run_session(graph, prover, rounds, verifier_rng) -> session_result
```

These are proposed interfaces, not an already implemented library. The verifier must not call the solver or the full-coloring checker to decide a protocol round.

Educational hash commitments

Use SHA-256 over an unambiguous encoding of a domain label, graph digest, session ID, round ID, vertex ID, permuted color, and a **fresh independent 32-byte nonce**. Use length-prefixed fields or a fixed canonical format. Keep the nonce secret until that vertex is opened. Never use Python's built-in `hash()` here. [6, 7]

`SHA256(color)` is easily searched: there are only three candidate colors. A random hidden nonce blocks that tiny dictionary demonstration. However, collision resistance alone is not a proof that this whole commitment construction is hiding. Present hash mode as an educational instantiation with stated assumptions.

For the demo, use `secrets` for nonces, secret palette permutations, and unpredictable verifier choices. For replayable experiments, separate seeded prover and verifier generators and clearly label their data **public simulation fixtures**. Do not claim secrecy for published seeds that reproduce witnesses or nonces. [6]

8. Experiments with a fixed stopping point

A **session** is t consecutive rounds and produces one accepted/rejected outcome. Repeat whole sessions independently. Do not count every round as an independent session observation.

Experiment	Configuration	Purpose
E1: honest control	Triangle; planted graphs with n = 12 and 24, cross-group edge probability 0.30, graph seeds 11, 22, 33; 100 sessions each, t = 20.	Check completeness and implementation correctness.
E2: false statements	K4 with L = 0, 6, 18 leaves; chosen assignment with b = 1; t = 1, 5, 10, 20, 40; N = 3,000 sessions per setting.	Compare measured rates with (1 - 1/m)^t. This is 15 settings and 45,000 sessions.
E3: bad witness	Corrupt one vertex in a planted valid coloring; compute the actual b and ensure b > 0; t = 1, 5, 20, N = 1,000 each.	Distinguish a bad assignment from a non-colorable graph.
E4: broken variants	Attempt a changed color or nonce after commitment; show the three-guess attack on unsalted color hashes.	Explain why binding and hiding are necessary.

Stop at the first rejection within each session; it cannot later become accepted. Keep planned round count t and executed round count separate. For E2 and E3, use the same base assignment throughout each setting, with new palette permutations and commitments every round.

Record enough to reproduce the figures

Save graph ID and digest, n, m, b, statement status, witness mode, t, N, accepted count, RNG mode and seeds for public fixtures, commit version, and environment versions. The controller may record b in evaluation data; the verifier must not receive it. Keep runtime measurements separate from graph generation and solving.

Statistics that match the question

The observed rate is accepted_count / N. Show **95% Wilson intervals** on measured rates. Compare with the predicted probability, and report the absolute difference. With N = 3,000, the worst-case standard error is about 0.0091, or 0.91 percentage points. [8, 9]

Zero observed acceptances does not mean zero risk. If all N independent sessions reject, a one-sided 95% binomial upper bound is 1 - 0.05^(1/N), approximately 3/N. At N = 3,000 this is about 0.10%. Do not rerun seeds until the plot looks right, and do not pool different graphs into one binomial interval.

9. Figures, checks, and the demo

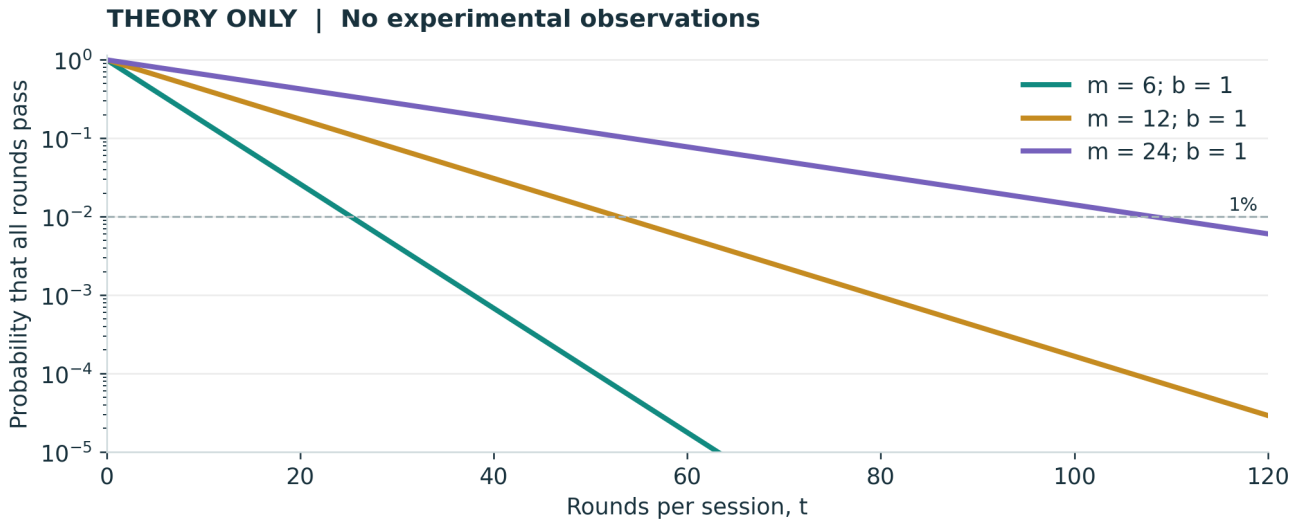


Figure 5. Planned reference curves only. In the final paper, add the actual measured points and Wilson intervals. Zero observations cannot be placed at zero on a log axis; use an upper-bound marker or a separate annotation.

Two main figures for the paper

1. **Acceptance versus rounds:** three theoretical curves for m = 6, 12, 24, overlaid with experimental points and intervals.

2. **The dilution effect at t = 20:** show the three K4-based graphs and their observed acceptance rates beside the predictions. Use the same explanation for a nontechnical reader.

Meaningful correctness checks

• Triangle is colorable; K4 is not; enumerate all 81 K4 assignments to confirm the minimum b is one.

• A solver timeout returns UNKNOWN. Invalid imports and invalid color labels are rejected.

• Every edge of a known honest example passes when deliberately challenged in a fresh round.

• Tampered openings and openings from a different vertex, round, or session are rejected.

• Commitments become fixed before the challenge; a second opening request for the same round is refused.

• Verifier input contains no full witness or unopened nonces. The demo's debug view is clearly labeled.

A 2-minute walkthrough

Show a proper coloring for teaching, switch to verifier view, seal the colors, reveal one selected edge, and explain why that single success is not enough. Switch to K4, show repeated sessions, then show the chart. Finish by explaining how extra valid edges dilute a fixed sampling budget. A notebook with clear cells and figures is sufficient.

10. Six weeks, three people

Choose owners by interest at the first meeting. Each person owns one area and reviews another. Everyone should be able to explain the basic protocol and the main chart.

Role	Main responsibility	Review responsibility
Arvin: coordinator and integration	Keep the question focused, connect protocol/math/experiments, maintain the README and paper narrative.	Review probability claims and public explanations.
Teammate A: graph and protocol lead	Graph utilities, small solver, prover/verifier, and core correctness checks.	Review the method description.
Teammate B: experiments and explanation lead	Run configurations, data summaries, plots, and demo notebook.	Check formulas using small examples.

This is a starting arrangement, not an assumption about anyone's ability. A teammate interested in proofs can take the proof appendix and share programming with Arvin. Avoid leaving one person with only formatting work.

Week	Shared objective	Concrete end-of-week checkpoint
1	Understand the puzzle; set up the repository; work through one round by hand.	Triangle and K4 fixtures, glossary, research question, agreed roles, small issues.
2	Generate/load graphs; validate assignments; implement tiny backtracking.	Graph module, TRUE/FALSE/UNKNOWN behavior, known witnesses, basic tests.
3	Implement commitments and the round/session state machine.	An honest run passes; K4 can be caught; changed openings fail; verifier view works.
4	Freeze the core method; run E1-E3 and broken-variant demonstrations.	Saved results, initial plots, formula comparison, recorded configuration.
5	Explain results; draft the short paper and 2-minute walkthrough.	A complete draft with limitations and two main figures. One teammate reproduces another's result.
6	Fix integration issues, simplify the explanation, and release.	Reproducible commands, final report, README, contribution record, v1.0 release.

Weekly rhythm: a short demo, one blocker per person, and a few issues that each fit in roughly 30-90 minutes. Use Discord for questions. Merge small changes. If someone misses a week, reduce optional work before increasing everyone's hours.

11. The paper and GitHub release

Working paper title: How Much Checking Is Enough? An Experimental Study of a Graph 3-Coloring Zero-Knowledge Protocol

Target **4-6 pages of main text**, plus references and an optional proof appendix. Use a simple student technical-report format. A GitHub release makes the report publicly accessible; it does not make it peer-reviewed.

Section	What to write
Abstract, 120-180 words	Question, method, one measured result when available, and the educational scope. Write this last.
1. Introduction	Explain private verification and the specific question about checking effort.
2. Background and protocol	Define graphs, witness, commitments, and one round. Cite the classic protocol.
3. Mathematical predictions	Completeness, fixed-assignment acceptance, false-statement bound, and K4 construction.
4. Methods	Graph families, prover modes, t and N, randomness, implementation, and interval method.
5. Results	Two figures, observed versus predicted values, deviations, and honest controls.
6. Discussion and limitations	Why b/m matters; limited graph families; hash assumptions; the distinction between testing and proving security.
References and contributions	Primary sources and a concrete account of each person's work.

Repository contents

Path	Purpose
`README.md`	Plain-language question, screenshot, quick start, main result, and report link.
`src/` and `tests/`	Reusable graph/protocol/experiment code and meaningful checks.
`examples/`	Public graph fixtures and clearly identified demonstration witnesses.
`experiments/` and `results/`	Frozen configuration, raw session summaries, and plots.
`notebooks/demo.ipynb`	The visual walkthrough.
`paper/`	Report source, PDF, figures, and references.
`CONTRIBUTIONS.md`	Individual work, design decisions, reviews, and acknowledgments.

Record tested package versions and exact commands. A fresh checkout should reproduce the main figures without manual edits. The release checklist should include a short description of educational limitations and an agreed license for the team's original code.

Make the project legible to admissions readers and HR

Lead with the question, a visual demo, and what each person built. After completion, a factual description could say: **“Collaborated on a three-person study of private verification, implementing a Python graph-coloring protocol and comparing randomized verification with mathematical bounds.”** Add actual numbers and your specific contribution only after they exist. Clear ownership and reproducible evidence make the work easier to assess.

12. Start small, then stop on purpose

First meeting: 40 minutes

1. **5 min:** agree on the question and the 3-4 hour weekly budget.

2. **10 min:** draw a triangle and K4; identify a witness and a bad edge.

3. **10 min:** simulate commitments with covered labels; explain why the edge comes last.

4. **10 min:** assign one small issue to each person and choose the next meeting.

5. **5 min:** say the difference between a bad witness, a false graph statement, and an implementation bug.

If the schedule slips

Keep K4 plus leaves, an honest control, the protocol, the theoretical comparison, and the report. Reduce graph variety and experimental repetitions with a stated reason. Defer adjacency matrices, elaborate animation, solver optimization, and any extra attack experiments. Do not drop error bars or silently replace UNKNOWN with FALSE.

Optional extensions: choose at most one

• **Math:** nonuniform edge challenges and how sampling weights change detection.

• **Programming:** a small interactive control panel inside the notebook.

• **Statistics:** first-detection time and its geometric distribution.

• **Cryptography:** implement an honest-verifier transcript simulator and compare its observable pair frequencies with real rounds; call this an illustration, not a security test.

References and reading route

The protocol and its assumptions were cross-checked against the original paper and university course materials; software and interval behavior were checked against official documentation. Some lecture notes contain typographical slips, so derive the probabilities from the edge count rather than copying isolated formulas.

1. Goldreich, Micali, and Wigderson (1991). Proofs that Yield Nothing But Their Validity or All Languages in NP Have Zero-Knowledge Proof Systems. Original protocol context; optional full reading.

2. Stanford CS 355 (2018). Interactive Proofs and Zero-Knowledge, Lecture 3. Start with Sections 3.1-3.2 for commitments and graph coloring.

3. Cornell CS 6830 (2011). Zero-knowledge proofs, Part 2. Cross-check of the protocol and repetition argument.

4. Boaz Barak. Zero knowledge proofs, Lecture 14. The simulation perspective.

5. NetworkX. greedy_color documentation. Heuristic behavior.

6. Python. secrets documentation. Randomness for security-sensitive choices.

7. Python. hashlib documentation. SHA-256 interface; not a proof of our commitment design.

8. NIST. Proportion confidence limits. Wilson and exact binomial intervals.

9. SciPy. Binomial proportion confidence intervals. Reproducible interval calculation.

Definition of done: one clean checkout, one reproducible experiment, one readable paper, and three people who can explain what they contributed.